Class Diagram0 2026/06/10

1 / 1

pkg

Business

+ getProductsForSale() : ProductForSale[]

+ getEmail() : String

+ getPhone() : String

+ getContactName() : String

+ getCif() : String

+ getCompanyName() : String

+ getProviderId() : int

+ <<constructor>> Provider(providerId: int, companyName: String, cif: String, contactName: String, phone: String, email: String, productsForSale: ProductForSale[]) () : void

- productsForSale : ProductForSale[]

- email : String

- phone : String

- contactName : String

- cif : String

- companyName : String

- providerId : int

Prov ider

Persistance

+ findById(clientId : int) : Client

+ save(client : Client) : void

+ findAll() : Client[]

<<interface>>
ClientDAO

Presentation

- showCart(client : Client) : void

- showProviderDetail(provider : Provider) : void

- handleProductPurchase(product : Product, providers : Provider[]) : void

- showProductDetail(product : Product) : void

- findProductsByProvider() : void

- findProductsByName() : void

- showUserProfile(client : Client) : void

- handleMainMenu(client : Client) : void

- handleRegister() : void

- handleLogin() : void

+ run() : void

+ <<constructor>> Controller(ui : UserInterface, clientManager : ClientManager, productManager : ProductManager, providerManager : ProviderManager, saleManager : SaleManager) : void

- ui : UserInterface

Controller

+ main(args : String[]) : void

Main

runs

Scanner

Exception

+ intInput(prompt : String) : int

+ stringInput(prompt : String) : String

+ showCartMenu() : void

+ showCart(cartLines : String[], totalPrice : float) : void

+ showProviderDetail(companyName : String, cif : String, contactName : String, phone : String, email : String, productLines : String[][]) : void

+ showProviderList(providerLines : String[]) : void

+ showProductDetail(productId : String, name : String, brand : String, model : String, providerLines : String[][]) : void

+ showProductList(productLines : String[]) : void

+ showUserProfile(clientId : int, fullName : String, phoneLines : String[], historyLines : String[]) : void

+ showMainMenu(fullName : String) : void

+ showAuthMenu() : void

+ showBanner() : void

+ <<constructor>> UserInterface() : void

- sc : Scanner

UserInterface

- ui

1 uses

uses

GeoApi

+ clear() : void

+ isEmpty() : boolean

+ getItems() : ProductForSale[]

+ remove(productId : String) : void

+ addItem(item : ProductForSale) : void

+ <<constructor>> ShoppingCart() : void

ShoppingCart

- COUNTRY : String

- POSTALCODE : String

- CITY : String

- STREET : String

- API_BASE : String

- API_KEY : String

Config

+ decreaseStock(units: int): void() : void

+ getUnitsInStock() : int

+ getSellingPrice() : float

+ getProviderId() : int

+ getProductId() : String

+ <<constructor>> ProductForSale(productId : String, providerId : int, sellingPrice : float, unitsInStock : int) : void

- unitsInStock : int

- sellingPrice : float

- providerId : int

- productId : String

ProductForSale

+ getLongitude() : double

+ getLatitude() : double

- longitude : double

- latitude : double

Geo

0..*

+ haversine(val : double) : double

+ calculateDistance(start : Geo, end : Geo) : double

+ getURLFromAddress(address : Address) : String

+ getGeolocationFromAddress(address : Address) : Geo

GeolocationApiManager

+ applyConstraintRules(total : float, geoManager :
GeolocationApiManager) : float

+ appliesVAT() : boolean

+ canBuy(product : Product) : boolean

+ getPhoneNumbers() : PhoneNumber[]

+ getName() : String

+ getClientId() : int

+ <<constructor>> Client(clientId : int, fullName :
String, phoneNumbers : PhoneNumber[]() : void

- phoneNumbers : PhoneNumber[]

- fullName : String

- clientId : int

Client

creates

+ getPostalCode() : String

+ getCountry() : String

+ getCity() : String

+ getNumber() : int

+ getStreet() : String

- postalCode : String

- country : String

- city : String

- number : int

- street : String

Address

+ getAddress() : Address

- address : Address

Inv alidAddressException

+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

+ getName() : String

+ getProductId() : String

+ <<constructor>> Product(productId : String, name : String) : void

- productName : String

- productId : String

Product

+ toString() : String

+ getNumber() : String

+ getCountryPrefix() : String

+ <<constructor>> PhoneNumber(countryPrefix : String, number : String) : void

- number : String

- countryPrefix : String

PhoneNumber

1..*

has

+ getPurchaseDate() : Date

+ getPricePaid() : float

+ getProductId() : String

+ getClientId() : int

+ <<constructor>> Sale(clientId : int, productId : String, pricePaid : float, purchaseDate : Date) : void

- purchaseDate : Date

- pricePaid : float

- productId : String

- clientId : int

Sale

+ generateNextClientId() : int

+ logout() : void

+ getPurchaseHistory(clientId : int) : Sale[]

+ getCurrentClient() : Client

+ register(fullName : String, phones : PhoneNumber[]) : Client

+ login(clientId: int): Client() : void

+ <<constructor>> ClientManager(clientDAO : ClientDAO, saleDAO : SaleDAO) : void

ClientManager

1

manages

1 - clientManager

uses

uses

+ getAllProducts() : Product[]

+ findById(productId : String) : Product

+ findByName(productName : String) : Product[]

+ <<constructor>> ProductManager(productDAO : ProductDAO) : void

ProductManager

- productManager 1

uses

uses

- findProductOrNull(productId : String) : Product

- createSale(client : Client, item : ProductForSale) : Sale

+ checkout(client : Client) : void

- calculateItemPrice(item : ProductForSale, client : Client) : float

+ calculateTotal(client : Client) : float

+ removeFromCart(productId : String) : void

+ addItem(item : ProductForSale, client : Client, product : Product) : void

+ getCart() : ShoppingCart

+ <<constructor>> SaleManager(saleDAO : SaleDAO, providerDAO : ProviderDAO, productManager : ProductManager, geolocationApiManager : GeolocationApiManager) : void

SaleManager

- saleManager 1

uses

1

cart

knows

knows

finds

uses

1

uses

+ getProductsForProvider(providerId : int) : ProductForSale[]

+ findOffer(providerId : int, productId : String) : ProductForSale

+ findByProductId(productId : String) : Provider[]

+ findById(providerId : int) : Provider

+ getAllProductsForSale() : Provider[]

+ <<constructor>> ProviderManager(providerDAO : ProviderDAO) : void

Prov iderManager

- providerManager
1

uses

uses

BusinessExceptions

+ applyConstraintRules(total : float, geoManager : GeolocationApiManager) : float

+ canBuy(product : Product) : boolean

+ getShippingAddress() : Address

+ getEmail() : String

+ <<constructor>> OnlineClient(clientId : int, fullName : String, phoneNumbers : PhoneNumber[], email : String, shippingAddress : Address) : void

- shippingAddress : Address

- email : String

OnlineClient

BusinessException

uses

1

shippingAddress

+ appliesVAT() : boolean

+ canBuy(product : Product) : boolean

+ getShippingAddress() : Address

+ getBillingAddress() : Address

+ getContactPersonName() : String

+ getNif() : String

+ <<constructor>> CorporateClient(clientId : int, fullName : int, String : int, phoneNumbers : PhoneNumber[], nif : String, contactPersonName : String, billingAddress :
Address, shippingAddress : Address) : void

- shippingAddress : Address

- billingAddress : Address

- contactPersonName : String

- nif : String

CoorporateClient

ClientNotFoundException

1

shippingAddress

1

billingAddress

ProductNotFoundException

+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

+ hasUvFilter() : boolean

+ getNumberOfLenses() : int

+ getPrescription() : double

+ <<constructor>> ContactLens(productId : String, name : String, brand : String, model : String, prescription : double,
numOfLenses : int, uvFilter : boolean) : void

- uvFilter : boolean

- numberOfLenses : int

- prescription : double

ContactLens

OutOfStockException

+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

+ expiresInLessThanThreeMonths() : boolean

+ containsPreservatives() : boolean

+ getExpirationDate() : LocalDate

+ getVolumeMl() : double

+ <<constructor>> Consumables(productId : String, name : String, brand : String, model : String, volumeMl : double, expirationDate : LocalDate, containsPreservatives : boolean)

- containsPreservatives : boolean

- expirationDate : LocalDate

- volumeMl : double

Consumables

+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

+ getCompletionHours() : double

+ <<constructor>> Services(productId : String, name : String, completionTimeHours : double) : void

- completionTimeHours : double

Serv ice

+ calculatePrice(sellingPrice : float, applyVAT : boolean) : float

+ getModel() : String

+ getBrand() : String

+ <<constructor>> PhysicalProduct(productId : String, name : String, brand : String, model : String) : void

- model : String

- brand : String

PhysicalProduct

+ <<constructor>> Glasses(productId : String, name : String, brand : String, model :
String) : void

Glasses

+ <<constructor>> InPersonClient(clientId : int, fullName : String, phoneNumbers : PhoneNumber[]() : void

InPersonClient

ApiHelper

+ updateStock(providerId : int, productId : String, units : int) : void

+ findByProductId(productId : String) : Provider[]

+ findById(providerId : int) : Provider

+ findAll() : Provider[]

<<interface>>
Prov iderDAO

ApìHelperExceptions

creates

- client : HttpClient

ApiHelper

+ deleteFromUrl(url : String) : String

+ postToUrl(url : url: String, body : String) : String

+ getFromUrl(url : String) : String

ApiHelperException

1

manages

1

manages

throws

+ findByClientId(clientId : int) : Sale[]

+ save(sale : Sale) : void

<<interface>>
SaleDAO

ConnectionInterruptedException

creates

1

manages

MalformedUrlException

1

manages

UnreachableServ erException

StatusExceptions + findByName(name : String) : Product[]

+ findById(productId : String) : Product

+ findAll() : Product[]

<<interface>>
ProductDAO

StatusException

creates

1

manages

IncorrectRequestException

PersistenceExceptions

+ findById(clientId : int) : Client

+ save(client : Client) : void

+ findAll() : Client[]

- persistClientToJSON(clients : Client[]) : void

- retrieveClientFromJSON(obj : JSONObject) : Client

- readJSONArray() : JSONArray

- FILE_PATH : String

ClientJSONDAO

PersistenceException

StatusCodeException

implements

throws

+ updateStock(providerId : int, productId : String, units : int) : void

+ findByProductId(productId : String) : Provider[]

+ findById(providerId : int) : Provider

+ findAll() : Provider[]

- persistProviderToJSON(providers : Provider[]) : void

- retrieveProviderFromJSON(obj : JSONObject) : Provider

- parseProductForSale(obj : JSONObject, providerId : int) : ProductForSale

- readJSONArray() : JSONArray

- FILE_PATH : String

Prov iderJSONDAO

Unav ailableApiException

implements

+ findByName(name : String) : Product[]

+ findById(productId : String) : Product

+ findAll() : Product[]

- retrieveProductFromJSON(obj : JSONObject) : Product

- readJSONArray() : JSONArray

- FILE_PATH : String

ProductJSONDAO

implements

throws

+ findByClientId(clientId : int) : Sale[]

+ save(sale : Sale) : void

- persistSaleToCSV(sale : Sale) : void

- retrieveSaleFromCSV() : List

- FILE_PATH : String

SaleCSVDAO

throws

implements

throws

+ findById(clientId : int) : Client

+ save(client : Client) : void

+ findAll() : Client[]

- readAll() : Client[]

- groupId : String

- BASE_URL : String

ClientAPIDAO

PersistenceWriteException

implements

1

calls

PersistenceFileNotFoundException

+ updateStock(providerId : int, productId : String, units : int) : void

+ findByProductId(productId : String) : Provider[]

+ findById(providerId : int) : Provider

+ findAll() : Provider[]

- BASE_URL : String

Prov iderAPIDAO

1

calls

implements

+ findByName(name : String) : Product[]

+ findById(productId : String) : Product

+ findAll() : Product[]

- BASE_URL : String

ProductAPIDAO

1

calls

implements

+ findByClientId(clientId : int) : Sale[]

+ save(sale : Sale) : void

- BASE_URL : String

SaleAPIDAO

1

calls

implements